



PACKAGING, GITOPS & EXTENDING KUBERNETES

Book 08 of the series — the delivery layer: Helm decoded (the four commands you ran blind in Book 07), Kustomize overlays, teaching the API new kinds with CRDs, Operators with domain knowledge, admission webhooks as the last checkpoint, and GitOps — the reconciliation loop aimed at your entire delivery system.



Helm

charts · releases



Kustomize

overlays



CRDs

new API kinds



Operators

runbooks as code



Webhooks

admission



GitOps

the pull model



ArgoCD

· Flux



Reconcile

the series' spine

PART I · HELM

- ☐ **CH 01** Helm — The Package Manager p.3
- ☐ **CH 02** Charts, Values & Templating p.4
- ☐ **CH 03** Releases — Lifecycle & Rollback p.5

PART II · KUSTOMIZE

- ☐ **CH 04** Kustomize — Overlays Without Templates .. p.6

PART III · EXTENDING THE API

- ☐ **CH 05** CRDs — Teaching the API New Kinds p.7
- ☐ **CH 06** Operators — Runbooks as Code p.8
- ☐ **CH 07** Admission Webhooks — The Last Gate p.9

PART IV · GITOPS

- ☐ **CH 08** GitOps — The Pull Model p.10
- ☐ **CH 09** ArgoCD & Flux — Concept Level p.11

PART V · LABS

- ☐ **LAB 1** Helm Lifecycle — Upgrade & Rollback p.12
- ☐ **LAB 2** Kustomize Overlays p.13
- ☐ **LAB 3** Build Your Own API Kind p.14

PART VI · PROVE IT

- ☐ **QUIZ** Self-Check Quiz p.15
- ☐ **KEY** Answer Key + Scoring p.16
- ☐ **GLOSS** Glossary p.17
- ☐ **CHEAT** One-Page Cheat Sheet p.18
- ☐ **FINAL** Where to Next → Book 09 p.19



Assumes Books 01–07. You've been *using* this book's contents all series without knowing: Book 07 · Lab 3 installed two charts, ServiceMonitor/VPA/HTTPRoute were CRDs (Books 07/06/03), ResourceQuota and PSS were admission controllers (Books 01/02), and cert-manager was an operator. Now the theory catches up.



Book 07 · Lab 3: you typed four Helm commands and got Prometheus, Alertmanager, Grafana, Loki and Promtail — **dozens of objects** — from four lines. Time to decode the machinery.

```
$ helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
#           ↑ the APP STORE           ↑ where the store lives
$ helm install mon prometheus-community/kube-prometheus-stack
#           ↑ RELEASE name           ↑ CHART from that repo
# one command → dozens of objects. how?
```

Chart — the *package*: a folder of templated YAML + default values. The recipe card.

Repository — the *store*: a URL serving charts. The cookbook shelf (apt/brew/homebrew vibes).

Release — an *installed instance* with a NAME and its own config. A cake you baked — receipt kept.



The problem Helm solves: raw YAML can't be parameterized. "Deploy Postgres, but with *my* password, *our* storage class, 3 replicas" without Helm = copy-paste-edit 500 lines and pray. With Helm: the chart holds templates, **you supply values**, the engine renders your personalized YAML. Same recipe, your kitchen.

Release names matter: Book 07's `mon` release named everything `mon-grafana`, `mon-kube-state-metrics` — one release prefix, zero collisions. Same chart installed 3× under 3 names = 3 independent stacks

See the receipt: `kubectl get secrets | grep sh.helm` — Helm stores release metadata in **Secrets** (Book 02's safe, hiding Helm's paperwork!)



One sentence: Helm is Kubernetes' **package manager** — *charts* are templated YAML packages from *repositories*, and *installing* one creates a named **release** whose every object is rendered from your values and tracked for later upgrade or rollback.

» Inside the recipe card — [CH 02: Charts, Values & Templating](#)

↑ back to top

```
mychart/
├─ Chart.yaml      # metadata: name, version, appVersion
├─ values.yaml     # DEFAULTS – the chart author's opinions
├─ templates/
│   ├─ deployment.yaml # Go-templated YAML
│   ├─ service.yaml
│   └─ _helpers.tpl    # reusable snippets (naming conventions...)
└─ charts/          # sub-charts (dependencies)
```

```
# templates/deployment.yaml – the templating you'll actually read:
metadata:
  name: {{ .Release.Name }}-web      # ← release name injected
spec:
  replicas: {{ .Values.replicaCount }}
  template:
    spec:
      containers:
        - image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
# .Values = merged values tree · .Release = install metadata
```

Values precedence — LAST WINS

Source	Wins over	Example
chart's <code>values.yaml</code>	— (the floor)	<code>replicaCount: 1</code>
your file <code>-f myvals.yaml</code>	chart defaults	<code>replicaCount: 3</code>
<code>--set replicaCount=5</code>	everything (the ceiling)	CI one-liners

```
$ helm install web ./mychart -f prod-values.yaml --set image.tag=v2.1
$ helm template ./mychart -f prod-values.yaml # render WITHOUT installing – review the YAML!
$ helm install web ./mychart --dry-run --debug # same idea, plus install mechanics
```



Why `helm template` **is a superpower:** the chart is a black box until you render it. Diff the output, read exactly what will hit your cluster (Book 01's `--dry-run` instinct, package-manager edition). Reviewing rendered YAML before apply is the mark of a Helm adult.



The values-check trap: `helm get values` shows only **YOUR overrides** — add `-a` (all) to see the computed values tree. "I set X but behavior didn't change" is usually a typo'd key silently ignored — `helm lint` and rendered-output review catch it.



One sentence: a chart = **templates + defaults**; installation = merging *your* values over the defaults (file beats default, `--set` beats file) and rendering plain YAML — which you can always preview with `helm template` before anything touches the cluster.

 The receipt book: upgrade, history, rollback — [CH 03: Releases](#)

↑ back to top



A release isn't a one-shot install — it's a **tracked lifecycle** with numbered revisions. If this feels familiar, it should: it's Book 01 · CH 21's rollout story, reborn at package level.

```
$ helm upgrade web bitnami/nginx --set replicaCount=3
$ helm history web
REVISION  STATUS      DESCRIPTION
1          superseded  Install complete
2          deployed   Upgrade complete
$ helm rollback web 1
$ helm history web
2          superseded  Upgrade complete
3          deployed   Rollback to 1 # ← NEW revision – history moves forward only!
```

Command	Job	Callback
<code>helm upgrade --install N chart</code>	create-or-update — idempotent ; the CI/CD workhorse	declarative wish, Book 01 style
<code>helm rollback N REV</code>	re-render an old revision as a new one	Book 01 · CH 21: undo ≠ rewind
<code>helm uninstall N</code>	delete the release's objects	namespaced cascade — Book 01 · Lab 3
<code>helm get values N</code> / <code>helm list</code>	audit config / inventory releases	—



The pro flags: `--atomic` — if an upgrade fails its health checks, **auto-rollback** (the Deployment-safety net from Book 01 · CH 20, applied to the whole package). `--wait` — block until all objects are ready. `helm upgrade --install --atomic --wait` is the CI one-liner worth memorizing.



Uninstall honesty: `helm uninstall` removes what Helm tracks — but **PVCs often survive** (Book 05: stateful resources are deliberately outside casual deletion). And the release receipts (those `sh.helm.release...` Secrets) go with it — uninstall = lose the ability to rollback. Book 09's **etcd backup** protects those receipts at a deeper layer.



One sentence: every install/upgrade/rollback is a numbered **revision** — forward-only history exactly like Deployment rollouts — and `upgrade --install` + `--atomic` turn "deploy a package" into a safe, idempotent, self-rescuing operation.

» The rival philosophy — [CH 04: Kustomize](#)

↑ back to top



Kustomize — Overlays Without Templates

6 MIN · CORE

Kustomize answers the same need as Helm — "same app, different environments" — with the **opposite philosophy**: no templating language. Your base YAML stays **valid, applicable, plain**; variants are declared as **patches and transformers** on top.

```

base/                                # the honest, deployable baseline
├─ deployment.yaml                  # real YAML – no {{ }} anywhere
├─ service.yaml
└─ kustomization.yaml              # resources: [deployment.yaml, service.yaml]
overlays/prod/kustomization.yaml
resources: [../../base]            # inherit the baseline
namespace: prod                    # TRANSFORMER: rewrite all namespaces
namePrefix: prod-                  # TRANSFORMER: prefix every name
images:
- { name: nginx, newTag: "1.26" }  # transformer: swap the tag
replicas:
- { name: web, count: 5 }           # transformer: scale

```

```

$ kubectl kustomize overlays/prod # PREVIEW – render the final YAML, change nothing
$ kubectl apply -k overlays/prod  # apply the overlay – BUILT INTO kubectl
$ kubectl get deploy -n prod
NAME      READY  IMAGE
prod-web  5/5    nginx:1.26 # prefix ✓ ns ✓ tag ✓ replicas ✓

```

	 Helm	 Kustomize
Mechanism	template + values → new YAML	patch + transform valid base YAML
Shines at	distributing software to strangers (charts)	managing YOUR env variants (dev/stage/prod)
Lifecycle	releases, rollback, repos	none — just rendering (GitOps-friendly!)
Install	separate binary	<code>kubectl -k</code> — already there



They compose: kustomizations can include `helmCharts:` entries — render a chart into your overlay pipeline. The real-world pattern: **Helm for third-party software** (postgres, prometheus), **Kustomize for your own** environment deltas. Rivals? No — different layers of the same delivery stack.



One sentence: Kustomize = **variations without templates** — a valid base, overlays that patch/transform (namespace, prefix, tags, replicas), rendered by `kubectl -k` itself; Helm distributes software, Kustomize manages *your* environments — and Lab 2 builds one live.

▶ Part III: teaching the API brand-new kinds — [CH 05: CRDs](#)

↑ back to top



You've spent seven books using kinds that **don't exist in core Kubernetes**: ServiceMonitor (Book 07), VPA (Book 06), Gateway & HTTPRoute (Book 03). Someone **taught the API those kinds** — with a Custom Resource Definition.

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: crontabs.stable.example.com # plural.group — THE API path
spec:
  group: stable.example.com
  versions:
  - name: v1
    served: true
    schema: # OpenAPI validation — BUILT INTO the API
      openAPIV3Schema:
        type: object
        properties:
          spec:
            type: object
            required: [cronSpec, image] # the API enforces this!
            properties:
              cronSpec: { type: string }
              image: { type: string }
  scope: Namespaced # or Cluster (Book 01 · CH 11 scopes!)
  names: { plural: crontabs, singular: crontab, kind: CronTab }
```

```
$ kubectl apply -f crontab-crd.yaml
$ kubectl get crontabs # ← the API now SPEAKS your kind. RBAC protects it,
# etcd stores it, kubectl explain crontab.spec works —
# first-class citizen, indistinguishable from built-ins.
```



The crucial boundary: a CRD is **schema-validated storage** — **nothing more**. Your CronTab gets validated, versioned, listed... and then **sits there**. No controller watches it, nothing schedules or runs. A CRD without a controller is a form nobody processes — the next chapter is the processor.

- CRDs are themselves cluster-scoped** (Book 01 · CH 11's list) — the definition governs the cluster; the *custom resources* (instances) live in namespaces per **scope**
- CRDs need installing before their resources** — apply order matters, and it's why Book 09's upgrade choreography and Book 08's sync waves (CH 09) exist



One sentence: a CRD extends the API Server with a **brand-new kind** — **validated, stored, RBAC-protected, kubectl-native** — but it's inert data until a controller gives it behavior; Lab 3 builds one and proves the API enforces *your* schema.

The controller half of the pattern — [CH 06: Operators](#)

↑ back to top



The series' spine returns one more time. Book 01's reconciliation loop — *watch actual, compare desired, act* — ran on built-in kinds. An **Operator** is that same loop, running on **your** CRD, with **your software's domain knowledge** encoded inside.

The Operator pattern

- CRD** e.g. kind: PostgresCluster (CH 05)
 - | human writes the WISH: replicas: 3, version: "16", backup: daily
 - ▼
- Operator** = custom controller (a Deployment) watching the CR
 - | reconciliation loop – with postgres expertise hardcoded:
 - ▼
- ACTS** via the k8s API: creates StatefulSet (Book 05!), headless Service (per-Pod DNS!), PVCs (volumeClaimTemplates!), configures replication, runs backups as CronJobs (Book 06!)
 - ▼
- reports status back onto the CR – Ready

✓ WHAT AN OPERATOR AUTOMATES

Everything a skilled DBA does at 3am **by hand**: bootstrap a cluster, add replicas, promote on failover, take consistent backups, rolling-upgrade postgres 15→16 with the right switchover order. The runbook, written in Go, executed forever.

OPERATORS YOU'VE ALREADY MET

prometheus-operator	taught the API ServiceMonitor (B07)
cert-manager	B03 · CH 06: "the cert just appears" – an operator for Certificate CRs
VPA	B06 · CH 08 – a CRD + controller
CloudNativePG, ECK...	database runbooks



Capability ladder (interview gold): operators evolve in phases — ① install the software, ② handle upgrades, ③ full lifecycle (backup, failover, scale). "Do we have an *operator* or just an *installer*?" is a serious procurement question — the word covers a spectrum.



One sentence: an Operator = **CRD + a reconciliation loop containing human expertise** — the same watch-compare-act spine from Book 01, now fluent in Postgres, certificates, or autoscaling; the CR is the wish, the operator is the eternal professional executing it.

The API's last checkpoint before etcd — [CH 07: Admission Webhooks](#)

↑ back to top



Every request this series has made — `kubectl apply`, controllers, operators — crosses the API Server's bouncer line: **AuthN** → **AuthZ (Book 02)** → **Admission** → **persist (etcd)**. Admission is the final, programmable checkpoint — and you've been living under it all along.

✓ BUILT-IN ADMISSION YOU'VE MET

```
ResourceQuota  B01 · CH 06 – "checked at the
                door, not on the floor" – admission!
PodSecurity    B02 · CH 13 – PSS enforce/warn/audit
                IS an admission controller
LimitRange     defaults injected per-pod
NamespaceLifecycle rejects objects into
                terminating namespaces (B01 · Lab 3)
```

🔧 EXTENSIBLE: CALL YOUR CODE

```
MutatingWebhookConfiguration
→ API Server calls your HTTPS service;
  webhook RETURNS A PATCHED object.
  (inject sidecars, defaults, labels)
ValidatingWebhookConfiguration
→ webhook says allow / deny+reason.
  ("no :latest tags" → 403 with
   your policy message)
```

The webhook flow

```
kubectl apply pod.yaml
```

- AuthN (who?) → AuthZ (may you?)
- **MUTATE** sidecar injected, resources defaulted
- **VALIDATE** sees the MUTATED object – order matters!
- etcd 📁

policy engines (Kyverno, OPA Gatekeeper) = prebuilt

validating/mutating layers driven by policy CRDs (CH 05 again!)



Two production warnings: (1) **failurePolicy** — `Fail` means: your webhook is down, *the entire API for those resources is down* (deploy nothing until the policy engine recovers). `Ignore` is available but means "policy offline = policy optional." (2) webhooks sit **in the write path** of everything — keep them fast, HA, and never co-locate them with the workloads they admit.



One sentence: admission is the **last programmable gate before etcd** — Quota and PSS were its built-in forms all series, and webhooks extend it with your own mutating/validating code, which is how house rules graduate from "convention" to "physically enforced at the door."

▶ Part IV: making Git the source of truth — [CH 08: GitOps](#)

↑ back to top



The classic delivery flow: CI runs `kubectl apply` or `helm upgrade`. Two quiet problems: your **pipeline holds cluster credentials** (Book 02's threat model, aimed at your own automation), and **drift is invisible** — someone hand-edits a Deployment, and Git says one thing while the cluster does another. Forever.

❌ PUSH (CI APPLIES)

```
CI —kubectl→ cluster
creds live in CI · hand-edits = silent drift ·
nothing reconciles · "who changed prod?"
```

✅ PULL (GITOPS)

```
Git ←pull— in-cluster AGENT —apply→ cluster
no creds flow OUT · agent reads Git (pull),
watches cluster, reconciles the diff ·
hand-edits get REVERTED (or alerted)
```



The four GitOps principles (OpenGitOps): ① the entire system is **declarative** (this whole series trained you for it) · ② desired state is **versioned in Git**, immutable history · ③ changes are **pulled** by an approved agent — nothing pushes in · ④ the agent **continuously reconciles**, not just at deploy time.

The spine, revealed: Book 01 taught *watch actual → compare desired → act* for Pods. Book 06 aimed it at replica counts. Book 08 aims it at **your entire cluster's configuration**: Git is the spec, the cluster is the reality, and the GitOps agent is the controller that cannot tolerate the difference. Drift is just another mismatch to reconcile.

Drift detection is the free lunch: even in manual-sync mode, the agent *shows* you every difference between Git and live — an audit trail and a safety net in one

What belongs in Git: everything you authored this series — Deployments, chart values, overlays, CRs, NetworkPolicies. What doesn't: true state (PV data) and raw Secrets (use external-secrets/SOPS — Book 02's encryption lesson, delivery edition)



One sentence: GitOps inverts the trust direction — **the cluster pulls its own desired state from Git and continuously reconciles toward it**, eliminating both the credentials-in-CI problem and silent drift with the series' oldest friend: the reconciliation loop.

The two graduated implementations — [CH 09: ArgoCD & Flux](#)

↑ back to top

Two CNCF-graduated implementations of the pull model, same reconciliation core, different personalities:

	🐙 ArgoCD	🍌 Flux
Personality	UI-first — watch diffs, click sync, app-of-apps dashboards	toolkit of lean controllers, no required UI
Extension model	Application CRD (yes — an operator whose CRD describes "keep ns X matching Git path Y"!)	GitOps Toolkit CRDs (Kustomization, HelmRelease, Source)
Know it for	visual drift triage, multi-cluster apps, sync waves/hooks	modularity, OCI artifact sources, bootstrap-from-nothing
Both	reconcile Git→cluster, detect drift, support Helm charts <i>and</i> Kustomize overlays natively (CH 03 + CH 04 compose!)	

```
# the ArgoCD Application — a CR describing one GitOps deployment:
apiVersion: argoproj.io/v1alpha1
kind: Application
spec:
  source:
    { repoURL: https://github.com/acme/fleet, path: apps/web/overlays/prod }
  destination:
    { server: https://kubernetes.default.svc, namespace: prod }
  syncPolicy:
    automated: { prune: true, selfHeal: true }
```

⚠️ **The two powerful switches, handled with respect:** `selfHeal` — manual `kubect1 edit` is **reverted automatically** (drift becomes impossible; so does your emergency hotfix unless it goes through Git — that's the deal). `prune` — objects deleted from Git are deleted from the cluster: powerful, and a Book 01-cascade-class decision. Turn both on *deliberately*, per app.

🌊 **Sync waves:** annotate objects `argocd.argoproj.io/sync-wave: "-1"` to order rollouts — **CRDs before CRs** (CH 05's apply-order rule, enforced), databases before the app that uses them (Book 05's sequence)

🔒 **Secrets in Git** stay encrypted (SOPS/sealed-secrets) or better, stay *out* of Git entirely via external-secrets — Book 02's layers, delivery edition

🔑 **One sentence:** ArgoCD and Flux are **operators for your fleet's configuration** — an Application CR declares "namespace X must match Git path Y," and automated sync + selfHeal + prune make Git's history and the cluster's reality the same document.

▶▶ Prove all of it with your hands — [Part V: Labs](#)

↑ back to top

**STEP 1 · Install — and name the release**

```
$ helm repo add bitnami https://charts.bitnami.com/bitnami && helm repo update
$ helm install web bitnami/nginx --set replicaCount=2
$ helm list
NAME      REVISION  STATUS    CHART          APP VERSION
web       1         deployed  nginx-x.y.z    1.25.x
$ kubectl get deploy # → web-nginx — RELEASE NAME prefixes everything (CH 01)
```

STEP 2 · Upgrade — history only moves forward

```
$ helm upgrade web bitnami/nginx --set replicaCount=3
$ helm history web
REVISION  STATUS    DESCRIPTION
1         superseded Install complete
2         deployed  Upgrade complete
$ helm rollback web 1 && helm history web
3         deployed  Rollback to 1 # NEW revision — Book 01 · CH 21's law
$ kubectl get deploy web-nginx # replicas back to 2 ✓
```

AUDIT THE RECEIPTS

```
$ helm get values web
USER-SUPPLIED VALUES:
replicaCount: 2 # your overrides only;
# add -a for the FULL tree (CH 02)
$ kubectl get secrets | \
  grep sh.helm.release
sh.helm.release.v1.web.v1...
# the receipts live in Secrets —
# Book 02's safe, holding Helm's
# paperwork (CH 01!)
```

SEE WHAT ONE COMMAND MADE

```
$ kubectl get all \
  -l app.kubernetes.io/instance=web
# deployment · service · (pods...) —
# one chart, many objects, all
# LABEL-tracked by release
# (Book 01 · CH 15's labels = the
# ownership mechanism!)
```

Proved: release naming, forward-only history, values auditing, and label-based ownership. **Teardown:** `helm uninstall web` (receipts go too — CH 03!).

[↑ back to top](#)



STEP 1 · An honest base

```
$ mkdir -p base overlays/prod && cd base
$ kubectl create deploy web --image=nginx:1.25 --replicas=1 \
  --dry-run=client -o yaml > deployment.yaml
$ cat > kustomization.yaml <<EOF
resources: [deployment.yaml]
EOF
$ kubectl kustomize . # renders EXACTLY the base — no templating, just valid YAML
```

STEP 2 · The prod overlay — four transformers

```
$ cd ../overlays/prod
$ cat > kustomization.yaml <<EOF
resources: [../../base]
namespace: prod
namePrefix: prod-
images:
- { name: nginx, newTag: "1.26" }
replicas:
- { name: web, count: 3 }
EOF
$ kubectl kustomize . | grep -E 'name:image:replicas|namespace'
namespace: prod # transformed
name: prod-web # prefixed
image: nginx:1.26 # retagged
replicas: 3 # scaled
```

STEP 3 · Apply the overlay

```
$ kubectl create ns prod
$ kubectl apply -k overlays/prod
$ kubectl get deploy -n prod
NAME      READY  IMAGE
prod-web  3/3    nginx:1.26 # every transformer, live
```

Proved: base stays valid YAML; the overlay is pure declaration; `kubectl -k` needs no extra binary. **Teardown:** `kubectl delete ns prod` (cascade — Book 01!).

[↑ back to top](#)



STEP 1 · Teach the API CronTab (CH 05's CRD, condensed)

```
$ kubectl apply -f - <<EOF
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata: { name: crontabs.stable.example.com }
spec:
  group: stable.example.com
  versions:
  - name: v1
    served: true
    schema:
      openAPIV3Schema:
        type: object
        properties:
          spec:
            type: object
            required: [cronSpec, image]
            properties:
              cronSpec: { type: string }
              image: { type: string }
        scope: Namespaced
      names: { plural: crontabs, kind: CronTab }
EOF
```

THE API SPEAKS YOUR KIND

```
$ kubectl apply -f - <<EOF
apiVersion: stable.example.com/v1
kind: CronTab
metadata: { name: my-cron }
spec:
  cronSpec: "*/5 * * * *"
  image: my-cron-img:1.0
EOF
$ kubectl get crontabs
NAME      AGE
my-cron   5s # first-class!
$ kubectl explain crontab.spec
# schema docs – generated
# from YOUR openAPIV3Schema
```

THE API ENFORCES *YOUR* SCHEMA

```
$ kubectl apply -f - <<EOF
apiVersion: stable.example.com/v1
kind: CronTab
metadata: { name: bad }
spec: { image: x:1 }
EOF
The CronTab "bad" is invalid:
spec.cronSpec: Required value
# rejected at the door –
# YOUR required field, enforced
# by the API Server itself.
# (and: my-cron just SITS there –
# no controller. CH 06 = the fix)
```

Proved: your kind is validated, stored, listable, and `explain`-able — schema enforcement included; behavior still absent (that's an operator's job). **Teardown:** `kubectl delete crontab my-cron`; `kubectl delete crd crontabs.stable.example.com`.

↑ back to top



Q1. `helm install mon chart-x` creates...

- A. a chart B. a repository C. a named release with tracked revisions D. a namespace called mon

Q2. After `helm rollback web 1` from revision 3, `helm history` shows...

- A. history trimmed to one entry B. a NEW revision 4, "Rollback to 1" — forward-only history C. revision 1 re-marked deployed, 2–3 deleted D. an error — rollback is Deployment-only

Q3. Values precedence, last-wins:

- A. `--set < -f file < chart defaults` B. `chart defaults < --set < -f file` C. they merge randomly D. `chart defaults < -f file < --set`

Q4. Kustomize differs from Helm fundamentally because it...

- A. patches and transforms valid base YAML — no templating language B. requires a separate binary C. stores releases in Secrets D. only works with charts

Q5. A CRD *by itself* provides...

- A. a running controller B. schema-validated, RBAC-protected storage — with no behavior C. automatic backups D. a new namespace

Q6. The `ServiceMonitor` kind (Book 07) exists because...

- A. it's built into core k8s B. kubectl installed it C. the prometheus-operator taught the API via a CRD D. Grafana created it dynamically

Q7. Order at the API Server's admission line:

- A. mutating webhooks run BEFORE validating — validators see the mutated object B. validation first, then mutation C. they run in parallel D. admission runs after persistence

Q8. ResourceQuota and Pod Security Standards are examples of...

- A. operators B. CRDs C. built-in admission controllers D. CNI features

Q9. GitOps improves on CI-push mainly because...

- A. it's faster B. an in-cluster agent PULLS from Git — no external credentials, and drift is continuously reconciled C. it removes the need for Git D. it skips RBAC

Q10. ArgoCD's `selfHeal: true` means...

- A. probes are auto-configured B. broken Pods restart faster C. Git gets updated from the cluster D. manual cluster edits are automatically reverted toward Git

[↑ back to top](#)



Q	Ans	Why — one line
1	C	Install = named release instance, revisions tracked in Secrets — CH 01
2	B	Rollback re-renders as a new revision; history never rewinds — CH 03, Lab 1
3	D	Defaults lose, file beats them, --set beats everything — CH 02
4	A	No templates: transformers/patches over valid YAML, built into kubectl — CH 04
5	B	CRD = storage + validation only; Lab 3's CronTab sat there — CH 05
6	C	Every non-core kind you've used came from someone's CRD — CH 05/06
7	A	Mutation first, so policy validates what will actually persist — CH 07
8	C	"Checked at the door" — Quota (B01) and PSS (B02) ARE admission — CH 07
9	B	Pull direction kills credential exfil + drift — CH 08
10	D	selfHeal = reconcile live state back to Git; hotfixes must go through Git — CH 09

**9–10**

Ready for Book 09

**6–8**Re-read missed chapters' 
boxes**<6**

Re-run Labs 1–3, then retake



Missed Q5 or Q6? The CRD/Operator split is THE extension-model question — data vs behavior. Missed Q8? Realizing Quota and PSS were admission all along is this book's biggest "oh" — the built-in world already ran the pattern you're now extending.

[↑ back to top](#)



Chart — Helm package: templated templates/ + default values.yaml + Chart.yaml metadata.

Repository — URL serving charts; `helm repo add` subscribes you to the store.

Release — a named installed instance of a chart; every object labeled and tracked together.

Revision — numbered snapshot of a release state; upgrades and rollbacks append (forward-only).

values precedence — chart defaults < `-f` file < `--set`; last wins.

helm template / --dry-run — render before install; review the exact YAML that will hit the cluster.

--atomic / --wait — auto-rollback on failed upgrade / block until ready; the CI-safe install flags.

Kustomize — template-free customization: base (valid YAML) + overlays of patches/transformers; `kubectl -k`.

Transformers — namespace, namePrefix, images, replicas — declarative rewrites applied at render time.

CRD — CustomResourceDefinition: teaches the API a new kind (group/version/kind, schema, scope); cluster-scoped itself.

Custom Resource (CR) — an instance of a CRD; inert data without a controller.

Operator — CRD + custom controller: domain-expert reconciliation loop that manages the software's full lifecycle.

Operator capability levels — install → upgrades → full lifecycle (backup, failover, scale).

Admission controller — API Server checkpoint after AuthZ, before etcd; ResourceQuota, LimitRange, PodSecurity are built-ins.

MutatingWebhookConfiguration — HTTPS callout that patches objects (sidecar injection, defaults); runs before validation.

ValidatingWebhookConfiguration — HTTPS callout that allows/denies with a reason; sees the mutated object.

failurePolicy — Fail (webhook down = API for those resources down) vs Ignore (policy optional when offline).

GitOps — declarative state in Git, pulled and continuously reconciled by an in-cluster agent; no outbound credentials, drift auto-detected.

Drift — divergence between Git and live state; the mismatch a GitOps agent exists to eliminate.

Application CRD — ArgoCD's object: source (Git path) + destination + syncPolicy describing one managed deployment.

selfHeal / prune — revert manual edits toward Git / delete live objects removed from Git.

Sync wave — ArgoCD ordering annotation; CRDs before CRs, databases before apps.

[↑ back to top](#)

 **HELM — COMMANDS**

```
$ helm repo add X URL && helm repo update
$ helm install N chart [-f v.yaml] [--set k=v]
$ helm upgrade N chart [--install] [--atomic --wait]
$ helm rollback N REV # new revision!
$ helm history N · list · get values N [-a]
$ helm template ./chart -f v.yaml # preview
$ helm uninstall N # receipts gone too
```

KUSTOMIZE — SKELETON

```
base/kustomization.yaml:
resources: [deployment.yaml, service.yaml]
overlays/prod/kustomization.yaml:
resources: [../..base]
namespace: prod · namePrefix: prod-
images: [{name: nginx, newTag: "1.26"}]
replicas: [{name: web, count: 3}]
$ kubectl kustomize overlays/prod # preview
$ kubectl apply -k overlays/prod
```

CRD MINIMUM + WEBHOOK FLOW

```
kind: CustomResourceDefinition
spec: { group, versions[.schema,
        scope, names{plural,kind} }
# CRD = data only · +controller = Operator
flow: authn → authz → MUTATE → VALIDATE → etcd
Quota · LimitRange · PSS = built-in admission
Kyverno/Gatekeeper = policy CRDs on webhooks
failurePolicy: Fail = webhook down ⇒ API down
```

GITOPS — ARGOCD APPLICATION

```
spec:
  source: {repoURL, path}
  destination: {server, namespace}
  syncPolicy:
    automated: { prune: true,
                 selfHeal: true }
# in Git: everything you author
# NOT in Git: PV data, raw Secrets
# (SOPS / external-secrets instead)
```

FACTS WORTH A JOB OFFER

Helm = charts (templates+values) → releases (named, revisioned, receipts in Secrets); rollback appends a revision

Kustomize = transformers over valid base YAML; Helm distributes software, Kustomize manages your env deltas — they compose

CRD = validated storage; CRD + controller with domain knowledge = Operator (the series' reconcile loop, generalized)

ServiceMonitor, VPA, GatewayAPI, ArgoCD's Application — all CRDs; cert-manager — an operator

Admission is the last gate: Quota/PSS built-in, webhooks extend it; mutation precedes validation; failurePolicy Fail = dependency

GitOps: agent pulls from Git, reconciles continuously — selfHeal reverts hand-edits, prune deletes removed objects

[↑ back to top](#)



Books 01–07 built the machine and taught it to observe itself — Book 08 made its **entire lifecycle declarative and self-reconciling**.

THE DELIVERY LAYER – fully mapped:

└─ Helm	charts · values precedence · releases · rollback
└─ Kustomize	base + overlays · transformers · kubectl -k
└─ CRDs	your own kinds – validated by the API itself
└─ Operators	the reconcile loop with domain knowledge
└─ Webhooks	the last gate – Quota/PSS were built-in admission
└─ GitOps	Git = spec · agent = controller · drift = mismatch

✓ You can now...

Decode and safely operate any chart — upgrade with `--atomic`, audit values, roll back knowing history never rewinds

Structure a fleet repo: Helm for third-party software, Kustomize overlays for your environments

Extend the API with your own CRD and explain precisely what an operator adds to it

Read the API Server's full request path — `authn` → `authz` → `mutate` → `validate` → `etcd` — and place every Book 02 concept on it

Set up GitOps with eyes open: what `selfHeal` and `prune` really promise, and what never belongs in Git



Where to next — Book 09 · Cluster Lifecycle & Production Readiness — the finale: everything so far ran *inside* a working cluster. Book 09 builds and maintains the cluster itself: `kubeadm` installation, **etcd backup & restore** (where Helm's release receipts and every object you've ever applied actually live!), control-plane upgrade choreography, GKE/EKS/AKS differences, multi-cluster strategy — and the CKA/CKAD/CKS exam map for the whole series.

— End of Book 08 ·  —